ITDSA Project

Submitted by:

Sivuziwe Tsotsa
Student Number: EDUV4826020

Institution:

Eduvos Midrand

Programme:

Bachelor of Science in Computer Science

Module:

Information Technology and Data Systems Architecture (ITDSA)

Date of Submission:

25 August 2025

Contents

Question 1.1. Business Rule 4
1.2 Conceptual Design: Entity-Relationship Diagram (ERD) 5
1.3 Logical Design: Normalisation 6
1.4 Physical Schema Design Description 6
2.1 Relational Algebra 8
2.2. Advanced Data Modelling: EERM 9
2.3 Concurrency Control 10
Question3: Advanced and Modern Database Concepts 10
3.1 Denormalization for Business Intelligence 10
3.2 Graph Database Design 11
References 12

Question 1.1. Business Rule

1. A customer can place many orders, but each order is placed by only one customer (1:M).

a. This represents a one-to-many relationship between Customer and Order.

Implementation:

• The primary key in the Customer table is CustomerID (PK).

• The Order table includes OrderID (PK) and CustomerID (FK), where CustomerID references the Customer table.

Constraints:

• To ensure every order is associated with a customer, CustomerID in the Order table must be NOT NULL.

• Referential integrity ensures that no order can exist without a valid customer, maintaining consistency across tables (Elmasri and Navathe, 2016).

2. Each order can contain multiple products, and each product can appear in multiple orders (M:N resolved via Order Items).

a. Order and Product have a many-to-many relationship, resolved through a junction table called OrderItem. This table includes:

• OrderItemID (PK)

• OrderID (FK referencing Order)

• ProductID (FK referencing Product)

• Quantity

• UnitPrice

b. To prevent duplicate entries, a composite unique constraint on OrderID and ProductID can be applied.

• Validation checks should ensure that Quantity and UnitPrice are positive values (Coronel and Morris, 2015).

3. One supplier supplies a product, but a supplier can supply many products (1:M).

a. Supplier and Product have a one-to-many relationship. The Supplier table includes SupplierID as the primary key.

b. The Product table includes SupplierID as a foreign key referencing the Supplier table.

c. Every product must have a valid supplier; therefore, SupplierID in the Product table must be NOT NULL (Rob and Coronel, 2007).

4. Each product must have a unique name and price and cannot exist without a supplier.

a. Attribute-level constraints and referential integrity are enforced in the Product table:

• Name is UNIQUE and NOT NULL

• Price is NOT NULL with a CHECK (Price > 0)

• SupplierID is a foreign key and must be NOT NULL

b. These constraints prevent duplicate product names (e.g., two entries for "Wireless Mouse") and ensure that every product has a valid price and supplier (Date, 2019).

5. Customers must provide a valid email address, which must be unique across the system.

a. The Customer entity enforces data quality and uniqueness through the following constraints in the Customer table:

• Email is UNIQUE and NOT NULL

• Format validation is applied using CHECK (Email LIKE '%@%.%')

b. These constraints prevent duplicate accounts and support contractability and identity verification (Connolly and Begg, 2015).

1.2 Conceptual Design: Entity-Relationship Diagram (ERD)

Entities and Attributes

Entity	Attributes
Customer	CustomerID (Primary Key), Name, Email, Phone Number, Address
Supplier	SupplierID (Primary Key) Name, Contact Email, Phone Number
Product	ProductID (Primary Key), Name, Description, Price, SupplierID (Foreign Key)
Order	OrderID (Primary Key), Order Date, CustomerID (Foreign Key), Total Amount
Order Item	OrderItemID (Primary Key), OrderID (Foreign Key), OrderID (Foreign Key), ProductID (Foreign Key), Quantity, UnitPrice

Relationships and Cardinality

- **Customer→ Order,** Many orders can be placed by one customer, which means each order can be placed by only one customer
- **Order →Order Item(1:M):** many order items can contain one order. One specific order item belongs to many orders items
- **Product→ Order Item(1:M):** In many order items, one product may appear. One product item is referred to by each order item.
- **Supplier→Product(1:M):** Many products can be supplied by one supplier. One supplier only supplies each product.
- **Order→Product (M: N):** Many products can include many orders. Many orders can have many products appear in them.

1.3 Logical Design: Normalisation

ONF (Unstructured Data)

Customer Order = {Customer Name, Email, ProductName, Supplier Name, Quantity, Price, Order Date}

1NF (Atomic Values)

- Repeating groups must be removed
- Customers, Orders, Products, and Suppliers need to have separate tables created for each of them.

2NF (Remove Partial Dependencies)

- Attributes need to be fully dependent on the entire primary key
- Quantity and UnitPrice depend on both OrderID and ProductID are split into composite keys.

3NF (Remove Transitive Dependencies)

- Another non-key attribute must not depend on a non-key attribute.
- SupplierName should not be in Product unless referenced via SupplierID.

1.4 Physical Schema Design Description

Customers

Column Name	Data Type	Constraints
CustomerID	INT	PK, NOT NULL, AUTO_INCREMENT
Name	VARCHAR (100)	NOT NULL

Email	VARCHAR (100)	UNIQUE, NOT NULL
Phone Number	VARCHAR (20)	CHECK (Phone Number LIKE '0%')
Address	VARCHAR (255)	NOT NULL

Suppliers

Column Name	Data Type	Constraints
SupplierID	INT	PK, NOT NULL, AUTO_INCREMENT
Name	VARCHAR (100)	NOT NULL
Name	VARCHAR (100)	NOT NULL
Contact Email	VARCHAR (20)	UNIQUE, NOT NULL
Phone Number	VARCHAR (20)	CHECK (Phone Number LIKE '0%')

Products

Column Name	**Data Type**	**Constraints**
ProductID	INT	PK, NOT NULL, AUTO_INCREMENT
Name	VARCHAR (100)	UNIQUE, NOT NULL
Description	TEXT	NULLABLE
Price	DECIMAL (10,2)	CHECK(Price>0), NOT NULL
SupplierID	INT	FK→ Suppliers (SupplierID), NOT NULL

Orders

Column Name	**Data Type**	**Constraints**
OrderID	INT	PK, NOT NULL, AUTO_INCREMENT
Order Date	DATETIME	NOT NULL
CustomerID	INT	FK→ Customers (CustomerID), NOT NULL
Total Amount	Decimal (10,2)	CHECK (Total Amount >=0), NOT NULL

Order Items

Column Name	Data Type	Constraints
OrderItemID	INT	PK, NOT NULL, AUTO_INCREMENT
OrderID	INT	FK→ Orders (OrderID), NOT NULL
ProductID	INT	FK→ Products (ProductID), NOT NULL
Quantity	INT	CHECK(Quantity>0), NOT NULL
UnitPrice	Decimal (10,2)	CHECK(UnitPrice>0), NOT NULL

2.1 Relational Algebra

a. **Customers who placed at least one order, and the date of the order**
    a. Let:
        i. Customer (CustomerID, Name, Email, Address)
        ii. Order (OrderID, Order Date, CustomerID)

    Plaintext:

    π_Name, Email, Order Date (Customer ⋈ Order)

b. **Products in stock in more than one warehouse, with total quantity**
    a. Inventory (ProductID, Warehouse ID, Quantity Available)
    b. Product (ProductID, Name)

    Plaintext

    Temp ← Inventory

    Grouped ← γ_ProductID, COUNT(WarehouseID)→WarehouseCount, SUM(QuantityAvailable)→TotalQty (Temp)

    Filtered ← σ_WarehouseCount > 1 (Grouped)

    Result ← π_Name, TotalQty (Filtered ⋈ Product)

c. **Suppliers of products ordered more than 50 times**
    a. Order Item (OrderID, ProductID, Quantity Ordered)
    b. Product (ProductID, SupplierID)
    c. Supplier (SupplierID, Name)

Plaintext

Grouped ← γ_ProductID, SUM(QuantityOrdered)→TotalQty (OrderItem)

Filtered ← σ_TotalQty > 50 (Grouped)

Join1 ← Filtered ⋈ Product

Join2 ← π_Name (Join1 ⋈ Supplier)

d. **Products and warehouse IDs where stock<10 units**
    a. Inventory (ProductID, WarehouseID, QuantityAvailable)
    b. Product (ProductID, Name)

Plaintext

Filtered ← σ_ QuantityAvailable < 10 (Inventory)

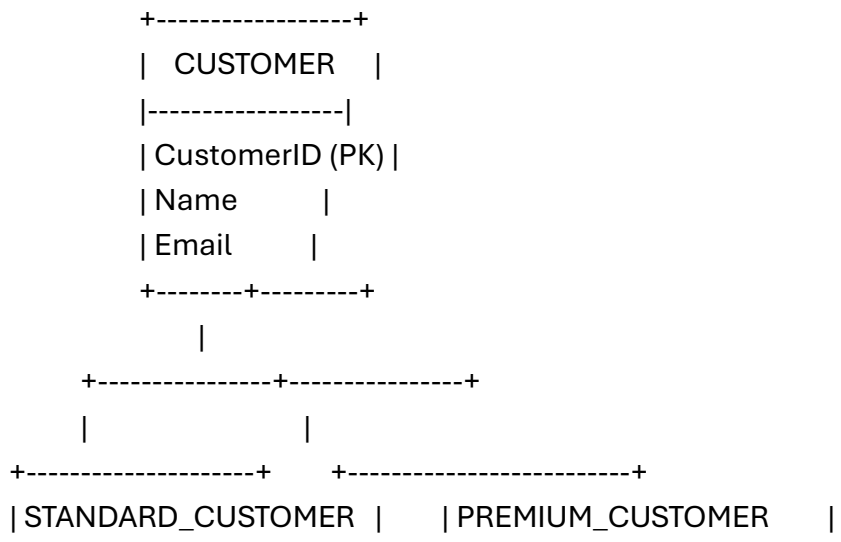
Result ← π_Name, WarehouseID (Filtered ⋈ Product)

2.2. Advanced Data Modelling: EERM

a. **Why EERM is suitable**
    a. **STANDARD_CUSTOMER and PREMIUM_CUSTOMER are** subtypes of the model CUSTOMER, which generalisation/specialisation allows.
        i. **Generalisation:** Common attributes are abstracted (e.g., Name, Email) into Customer
        ii. **Specialisation:** Subtype-specific attributes are added (e.g., Account Manager, Discount Rate for PREMIUM).
        iii. **Supertypes/Subtypes:** Exclusive participation is enabled through inheritance and enforcement (a customer is either standard or premium, not both).

Future extensibility is supported by this structure as it improves semantic clarity (e.g., adding a “Business Customer” subtype).

b. **Partial EERM Diagram**



|---------------------| |-------------------------|
CustomerID (PK, FK)		CustomerID (PK, FK)
		Account Manager
		Discount Rate
+---------------------+ +-------------------------+

- **Disjoint Specialisation:** A customer is either standard or premium
- **Exclusive Attributes:** Account Manager and Discount Rate are only for PREMIUM_CUSTOMER

2.3 Concurrency Control

c. **Concurrency Problems: Lost Update**
   a. **Problem:** One update overwrites the other if IT1 and IT2 interleave
      T1: READ(StockQuantity) → 15
      T2: READ(StockQuantity) → 15
      T1: StockQuantity = 15 - 1 → 14
      T1: WRITE(StockQuantity) → 14
      T2: StockQuantity = 15 + 50 → 65
      T2: WRITE(StockQuantity) → 65

      Incorrect Final Value:65
      Correct Final Value:15-1+50=64
      Lost Update: T1's update(14) is overwritten by T2's(65)

d. **Conflict-Serializable Schedule**
      T2: READ(StockQuantity) → 15
      T2: StockQuantity = 15 + 50 → 65
      T2: WRITE(StockQuantity) → 65
      T1: READ(StockQuantity) → 65
      T1: StockQuantity = 65 - 1 → 64
      T1: WRITE (Stock Quantity) → 64

Final Stock Quantity:64

T2→T1 is executed because of a conflict-serializable

Question3: Advanced and Modern Database Concepts

3.1 Denormalization for Business Intelligence

1.

CustomerID-Each customer has a unique identifier

Customer Name-The customer's full name

Email- Email address of the customer

Total Orders -The customer places a total number of orders

Total Amount Spent -The customer order items are totalled

2.

```
INSERT INTO CustomerSummary (CustomerID, CustomerName, Email,
TotalOrders, TotalAmountSpent)
SELECT
 c.CustomerID, c.Name, c.Email,
 COUNT(o.OrderID) AS TotalOrders,
 SUM(oi.Quantity * oi.Price) AS TotalAmountSpent
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
JOIN OrderItems oi ON o.OrderID = oi.OrderID
GROUP BY c.CustomerID, c.Name, c.Email;
```

3.2 Graph Database Design

1.

Graph databases like Neo4j excel at modelling relationships:

Natural Fit for Recommendations: As direct edges they represent "who brought what" and "what was brought together" as direct edges.

Efficient Traversal: Without expensive joins graph queries (e.g., Cypher) can quickly find connected

Flexible Schema: Without rigid table structures can easily evolve relationships and properties.

Real-Time Insights: Relationships dynamically for instant recommendations for traverse customer-product.

2.

Nodes:

- **Customer:**(CustomerID, Name, Email)
- **Product:**(ProductID, Name, Category, Price)

Relationships:

Relationship	From	To	Properties
BOUGHT	Customer	Product	Quantity, Purchase Date

ALSO_BOUGHT	Product A	Product B	Frequency, LastCoPurchaseDate

(Customer: Alice) —[BOUGHT {Qty:2}]—▶ (Product: Phone)

(Customer: Alice) —[BOUGHT {Qty:1}]—▶ (Product: Charger)

(Product: Phone) —[ALSO_BOUGHT {Freq:25}]—▶ (Product: Charger)

(Customer: Bob) —[BOUGHT]—▶ (Product: Phone)

(Customer: Bob) —[BOUGHT]—▶ (Product: Earphones)

(Product: Phone) —[ALSO_BOUGHT] —▶ (Product: Earphones)

References

- Connolly, T. and Begg, C., 2015. *Database Systems: A Practical Approach to Design, Implementation, and Management*. 6th ed. Pearson Education.
- Coronel, C. and Morris, S., 2015. *Database Systems: Design, Implementation, & Management*. 11th ed. Cengage Learning.
- Date, C.J., 2019. *Database Design and Relational Theory: Normal Forms and All That Jazz*. 2nd ed. O'Reilly Media.
- Elmasri, R. and Navathe, S.B., 2016. *Fundamentals of Database Systems*. 7th ed. Pearson.
- Rob, P. and Coronel, C., 2007. *Database Systems: Design, Implementation, and Management*. 8th ed. Thomson Course Technology.